

DUBLIN BUSINESS SCHOOL

MS Cybersecurity

NETWORK SYSTEMS & ADMINISTRATION

Class Assignment 1

GROUP – 4

Member 1: Hammad Malik

Member 2: Khawaja Abdul Moiz

Member 3: Muhammad Arshman Noor

Individual Report

Student Name: Muhammad Arshman Noor

Student Id: 20096163

Instructor: Prem Sagar Gompaa

Word Count: 2171

CONTENTS

Summary:	3
Tools Used:	3
1. Introduction:	4
2. Content:	5
2.1 Infrastructure Setup with Terraform:	5
2.2 Configuration Management	6
2.3 Docker Container Deployment	6
2.4 CI/CD Pipeline Integration	7
2.5 Final Deployment and Architecture	7
3. Architecture Diagram	8
4. Challenges and Solutions	9
4.1 Ansible Incompatibility with Windows	9
4.2 SSH Key Authentication Failure	9
4.3 GitHub Large File Size Limit	10
4.4 Docker Hub Authentication Error	10
4.5 CI/CD SSH Deployment Step Failure	11
5. Conclusions	12
6. References	13

Summary:

This report contains the design, implementation, challenges I faced, solutions and deployment of an automated containerized web application. This project involves Amazon Web Services, Elastic Compute Cloud virtual server using Terraform, Docker for containerizing and CI/CD pipeline on Github. A sample web application was successfully deployed and made publicly accessible via public IP address. This report shows the complete process of automating web application, challenges I faced, how I encountered them and Solutions with screenshots and a detailed architecture diagram.

Although, it was a group assignment but this is an individual report containing details only about the process of single member, challenges and solutions.

Tools Used:

1. Terraform
2. Amazon Web Services (AWS)
3. Docker
4. Docker Hub
5. GitHub
6. Python
7. Flask
8. VS Code
9. Amazon CLI

1. Introduction:

The adoptions of cloud computing and containerization technologies are increasing rapidly and has changed the way modern software applications are developed, deployed and managed. Organizations are increasing their rely on automated pipelines and Infrastructures as Code (IAC) tools to provision and configure cloud resources with speed, consistency and reliability.

The main objective of this assignment is to provision cloud infrastructure automatically using Terraform; configure a server environment to support Docker container, build and deploying a sample of Flask Web application as a Docker container, and implementation a CI/CD pipeline using GitHub Actions to automate the build and push process.

The chosen cloud provider is Amazon Web Services (AWS), utilizing eu-west-1 (Ireland) region.

The other tools selected to complete this task are Terraform for infrastructure provisioning, Docker and Docker Hub for containerization, GitHub Actions for CI/CD automation and Python Flask as the sample web application framework. This report presents the each phase of the project, discuss the rationale for tool selection, and reflect on the challenges faced during implementation and solutions.

2. Content:

2.1 Infrastructure Setup with Terraform:

The first stage of the project focused on setting up the cloud infrastructure needed to host the application server. For this, I chose Terraform, HashiCorp's open-source Infrastructure as Code tool. Terraform lets you define cloud resources declaratively using HCL (HashiCorp Configuration Language), which meant the infrastructure could be version-controlled, reproducible, and set up automatically rather than by hand (HashiCorp, 2024).

To get started, I installed and configured the AWS CLI on my local Windows machine, running `aws configure` to enter my AWS Access Key ID, Secret Access Key, default region (eu-west-1), and preferred output format. I then installed Terraform and added it to the system PATH so it could be run directly from the command line.

Next, I wrote a `main.tf` file to define a `t3.micro` EC2 instance based on an Ubuntu AMI. Alongside this, I configured a security group permitting inbound traffic on port 22 for SSH and port 80 for HTTP, plus an SSH key pair for accessing the server. I also embedded a `user_data` shell script in the configuration so Docker would install automatically the first time the instance booted.

With the configuration in place, I ran `terraform init` to pull down the AWS provider plugin, then used `terraform plan` to check what would be created before actually deploying anything. Running `terraform apply` provisioned the EC2 instance, security group, and key pair on AWS, and once it finished, the server's public IP address was displayed as output.

I did consider AWS CloudFormation as an alternative, since it's Amazon's own native IaC service. In the end, though, I went with Terraform because it isn't tied to a single cloud provider, has a cleaner syntax to work with, and benefits from a larger, more active community (Brikman, 2022). AWS CDK was another option I looked at, since it allows infrastructure to be written in general-purpose languages like Python or TypeScript rather than a dedicated config language.

2.2 Configuration Management

The second phase was about automatically setting up Docker on the server. My original plan was to use Ansible for this. Ansible is a configuration management tool that doesn't need any agent installed on the target machine, and it uses YAML playbooks to automate server tasks (Red Hat, 2024). The problem was that Ansible doesn't work properly as a control machine on Windows, and since I was developing on a Windows machine, this was a real issue.

So instead, I used Terraform's built-in `user_data` feature. This script ran automatically when the EC2 instance first started up. It updated the package manager, installed Docker, started the Docker service, and made sure Docker would start again automatically if the server rebooted. This gave me the same result I would have gotten with Ansible, but it kept everything inside Terraform instead of needing a separate tool.

If I'd been working on Linux or macOS instead of Windows, I would have used Ansible instead. It's idempotent (meaning you can run it repeatedly without causing problems), it uses a clear playbook structure, and it's easier to review and audit. One thing I could try in future is setting up WSL (Windows Subsystem for Linux), which would let me run Ansible directly from a Windows machine.

2.3 Docker Container Deployment

The third phase involved containerising the sample web application and deploying it on the server. Docker was selected as the containerisation platform due to its widespread adoption, portability, and ability to create isolated and reproducible application environments (Docker Inc., 2024).

A simple web application was developed using Python Flask framework. The application serves an HTML page displaying the module code, group number, and member details including student IDs. Three files were created for the application: `app.py` containing the Flask application code, `requirements.txt` listing Flask as the only dependency, and a `Dockerfile` defining the container image.

The base image used in the `Dockerfile` was `python:3.11-slim`, chosen for its small footprint. The `Dockerfile` copied the application files into the container, installed Flask via `pip`, exposed port 80,

and defined the command to start the Flask server. The image was built locally using docker build, tested at localhost:80 in the browser, and then tagged and pushed to Docker Hub.

Docker Hub served as the container registry for storing and distributing the image. An alternative registry would have been Amazon ECR (Elastic Container Registry), which offers tighter AWS integration and IAM-based access control. However, Docker Hub was preferred for its simplicity and free tier. The container was deployed on the server using EC2 Instance Connect, with docker run commands specifying port mapping and a restart policy.

2.4 CI/CD Pipeline Integration

To automatically build and push the Docker image every time I push code to GitHub a CI/CD pipeline was setup. To set it up, I went to GitHub Actions and uses YAML for configuration.

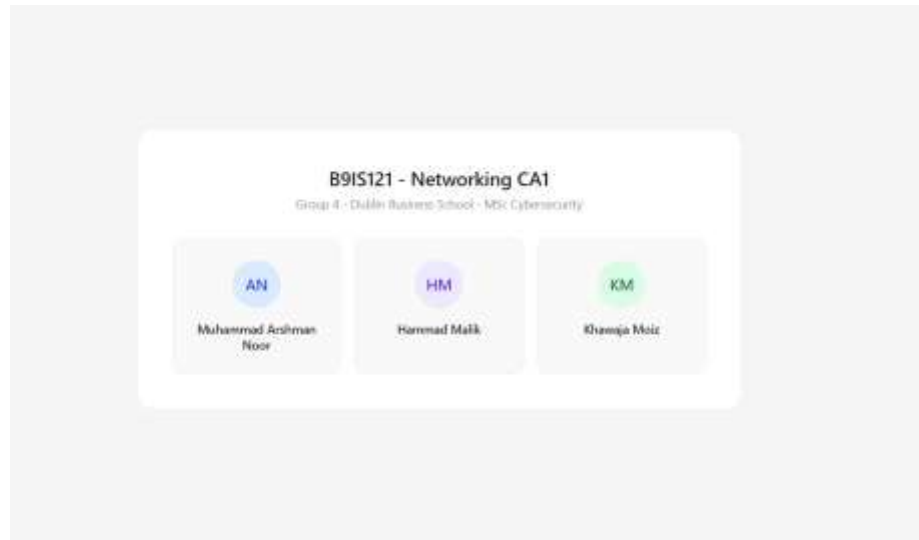
Inside the .github/workflows folder I created a workflow file. To automatically run on every push to main branch the pipeline was set up, and it had three main steps; checking the repository code, logging into Docker Hub using stores secrets, and building and pushing the Docker image with the docker/build-push-action. I set my Docker Hub username and access token as repository secrets so the credentials would not be exposed anywhere in the codebase.

The pipeline ran successfully in about 30-40 seconds each time, which confirms that the Docker image was being built and pushed to Docker Hub automatically with every commit.

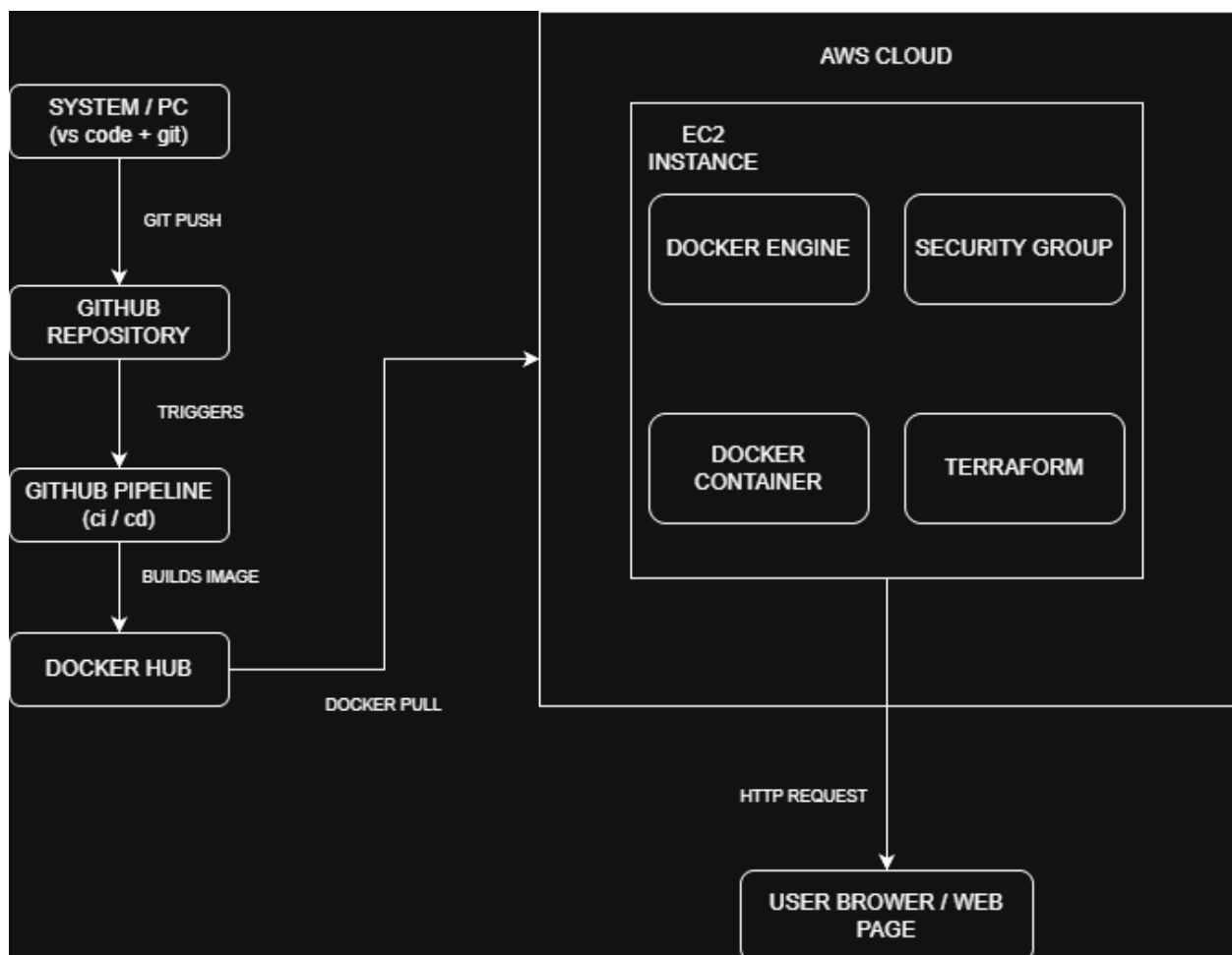
2.5 Final Deployment and Architecture

The final phase was to check everything works perfectly. I opened the web app in the browser using server's public IP address when the container was up and running on EC2 server. The page was loaded perfectly and showing group details, which means the deployment of pipeline was working perfectly.

I write code locally in VS code and push it to the GitHub by using Git. Every push automatically triggers the GitHub Actions Pipeline, which builds the Docker image and pushes it to Docker Hub. The EC2 instance runs Docker and pulls down the latest image. The container runs the flask and make it publicly accessible by server's IP address.



3. Architecture Diagram

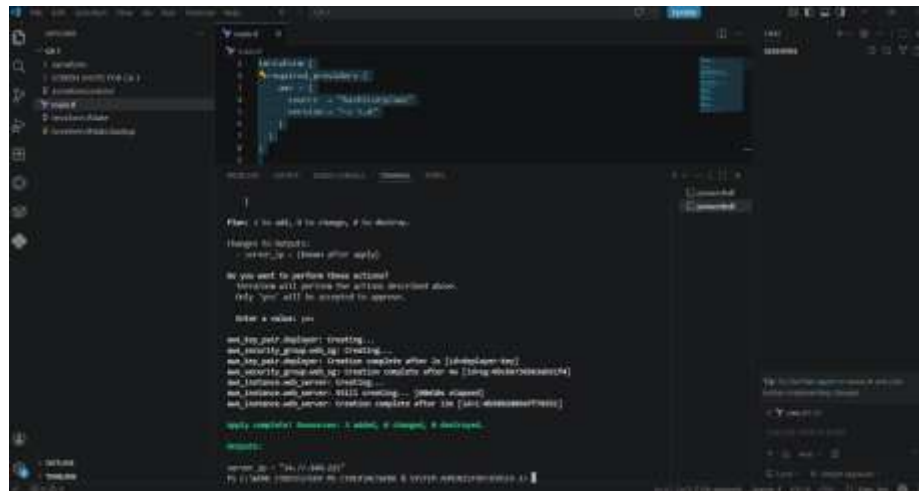


4. Challenges and Solutions

Several challenges were faced during the implementation of this project.

4.1 Ansible Incompatibility with Windows

The biggest and first challenge I faced was installing an ansible. Ansible works on MacOS and Linux only. But I wanted to do it directly on my windows, so I replaced Ansible with Terraform's `user_data` feature, which gave me the same result by automating the server setup at boot time.



4.2 SSH Key Authentication Failure

A second challenge I faced was configuring the SSH key pair for server access. Multiple attempts to connect to the EC2 instance via SSH resulted in 'Permission denied' errors. This error occurs due to a mismatch between the key registered with AWS and private key available locally. This was resolved by updating the terraform configuration to reference new public key by updating it with a new RSA key pair using ssh-keygen. The server then destroyed the old terraform and applied a new one.

```
Deploy to server via SSH 1s
1 ▶ Run appleboy/ssh-action@master
19 ▶ Run echo "$GITHUB_ACTION_PATH" >> $GITHUB_PATH
24 ▶ Run entrypoint.sh
67 Downloading drone-ssh-1.8.2-linux-amd64 from https://github.com/appleboy/drone-ssh/releases/download/v1.8.2
68 ===== CLI Version Information =====
69 Drone SSH version 1.8.2
70 =====
71 2025/06/28 09:43:53 ssh: handshake failed: ssh: unable to authenticate, attempted methods [none publickey], no supported methods remain
72 Error: Process completed with exit code 1.
```

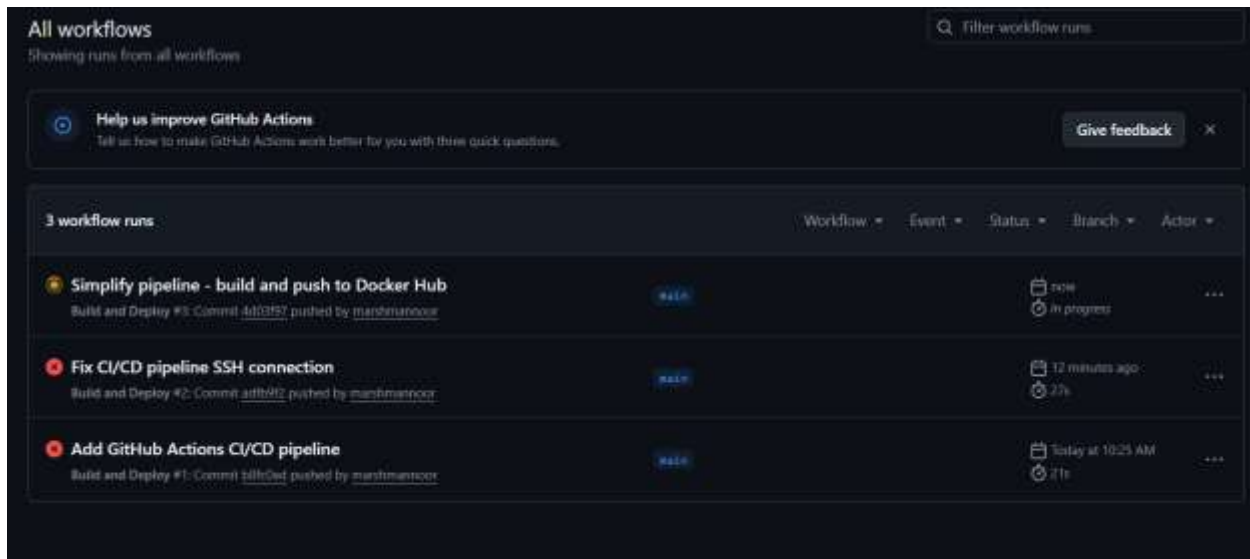
4.3 GitHub Large File Size Limit

A third challenge was occurred while pushing files to GitHub. The terraform directory size was 685 MB and GitHub file size limit was 100 MB, and because of this the push was rejected. This was resolved by applying the git filter-repo tool to remove the terraform directory from the entire Git history, and adding it to a .gitignore, to prevent it from being tracked in future commits.

```
login canceled
PS E:\WORK CYBER\STUDY MS CYBER\NETWORK & SYSTEM ADMINISTRATION\CA 1> docker tag my-web-app manoor/my-web-app:latest
PS E:\WORK CYBER\STUDY MS CYBER\NETWORK & SYSTEM ADMINISTRATION\CA 1> docker push manoor/my-web-app:latest
The push refers to repository [docker.io/manoor/my-web-app]
8679eb52b786: Waiting
655c79c4cbcb: Waiting
3250ea7ceadc: Waiting
4668add98893: Waiting
72c03230f136: Waiting
8bd2cf7d3f6a: Waiting
b39477d3277a: Waiting
0c1d0129248c: Waiting
8c471a93fc52: Waiting
push access denied, repository does not exist or may require authorization: server message: insufficient_scope: author
ization failed
PS E:\WORK CYBER\STUDY MS CYBER\NETWORK & SYSTEM ADMINISTRATION\CA 1> █
Ln 4, Col 36
```

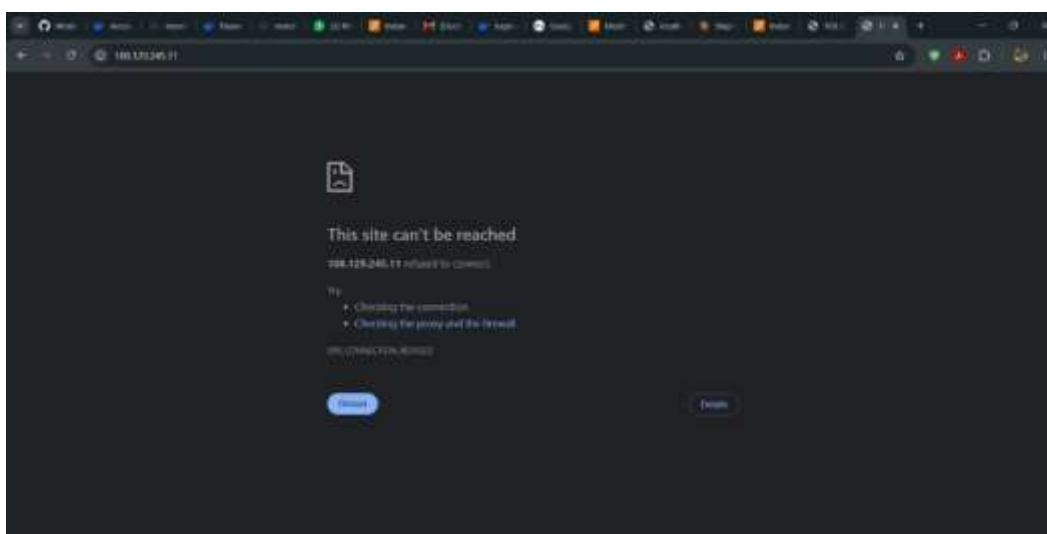
4.4 Docker Hub Authentication Error

A fourth challenge, I faced was about CI/CD pipeline configuration. I was using an incorrect username which results into failure of Docker Hub authentication, which was causing the pipeline in GitHub Actions to fail with an ‘unauthorized, incorrect username or password’ error. This was resolved by verifying the correct username on Docker Hub website and updating it in GitHub repository secrets accordingly.



4.5 CI/CD SSH Deployment Step Failure

A fifth challenge I faced was the failure of SSH-based deployment in the GitHub Actions pipeline. Even after adding the SSH private key as a GitHub, the pipeline was not able to authenticate with the EC2 server via SSH. This was caused by the key format incompatibility between the locally generated key and the format expected by the apple/ssh-action. The SSH deployment step was removed from the pipeline, and leaving the pipeline to handle only the build and push stages. The container on the server was performed manually by using EC2 instance connect. This is noted as an area for future improvement.



5. Conclusions

This project demonstrates the integration of multiple DevOps automation tools to deploy a web application on cloud infrastructure. For AWS provisioning, Terraform proved to be an effective and reliable tool in a declarative and reproducible manner. Docker simplified the application packaging and deployment by ensuring the application ran consistently regardless of underlying environment. GitHub Actions provided a straightforward and efficient CI/CD pipeline that automated the image build and push process with minimal configuration.

To enhance this deployment in a production context several improvements could be made. Firstly, implementing HTTPS using a TLS/SSL certificate from AWS Certificate Manager or Let's Encrypt would secure data in transit. Secondly, using Amazon ECS or Kubernetes for container coordination would improve scalability and fault tolerance. Thirdly, resolving the SSH-based deployment step in the GitHub Actions pipeline would enable fully automated end-to-end deployment, eliminating the need to manually update the running container. Fourthly, storing the Terraform state file in an AWS S3 bucket with DynamoDB state locking would enable collaborative infrastructure management. Finally, using Ansible from WSL environment would allow proper configuration management from Windows machines in future projects.

Overall, this project provides valuable hands-on experience with modern cloud automation tools and DevOps workflows, reinforcing the importance of Infrastructure as Code, containerization, and CI/CD practices in contemporary system administration and cloud engineering.

6. References

1. Amazon Web Services (2024) *AWS EC2 Documentation*. Available at: <https://docs.aws.amazon.com/ec2> (Accessed: 1 July 2026).
2. Brikman, Y. (2022) *Terraform: Up and Running*. 3rd edn. Sebastopol: O'Reilly Media.
3. Docker Inc. (2024) *Docker Documentation*. Available at: <https://docs.docker.com> (Accessed: 1 July 2026).
4. GitHub (2024) *GitHub Actions Documentation*. Available at: <https://docs.github.com/en/actions> (Accessed: 1 July 2026).
5. HashiCorp (2024) *Terraform Documentation*. Available at: <https://developer.hashicorp.com/terraform/docs> (Accessed: 1 July 2026).
6. Red Hat (2024) *Ansible Documentation*. Available at: <https://docs.ansible.com> (Accessed: 1 July 2026).
7. Turnbull, J. (2023) *The Docker Book*. Available at: <https://dockerbook.com> (Accessed: 1 July 2026).